NYC 311 Data Curation Project

ITAI 3375 — Curating Data for AI Optimization

This notebook supports the final presentation by creating:

- A reproducible NYC 311 data subset
- Raw and curated data profiles
- A documented curation pipeline
- Privacy-aware location handling
- Preliminary urgency labels for human review
- Five before-and-after quality scores
- A presentation-ready quality chart
- Curated CSV and 50-record annotation review sheet

Important: The urgency labels are preliminary rules, not emergency or agency decisions.
The **Accuracy** result is clearly labeled as an internal logic proxy because true accuracy requires external ground truth.

```
# Run this once in Google Colab
!pip -q install pandas numpy matplotlib requests
```

Complete reproducible project code

```
# NYC 311 Data Curation Project
# ITAI 3375 — Curating Data for AI Optimization
#
# This script supports the final presentation:
# 1. Downloads a reproducible subset from NYC Open Data
# 2. Profiles the raw data
# 3. Cleans and standardizes the selected fields
# 4. Applies privacy-aware location handling
# 5. Creates preliminary urgency labels for human review
# 6. Calculates five before/after quality dimensions
# 7. Saves a chart, curated CSV, annotation sample, and score table
#
# IMPORTANT:
# - Change TARGET_ROWS to the exact number used in your project.
# - The "Accuracy" score below is an INTERNAL LOGIC PROXY, not verified
#   real-world ground-truth accuracy.
# - The urgency rules are preliminary annotations and should not replace
#   human or agency decision-making.

from __future__ import annotations

import math
import re
import time
from pathlib import Path
from typing import Iterable

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests


# ============================================================
# 1. PROJECT CONFIGURATION
# ============================================================

DATASET_ID = "erm2-nwe9"
API_URL = f"https://data.cityofnewyork.us/resource/{DATASET_ID}.json"

# Set this to the exact record count used in your project.
```

```
# 100,000 is a practical Colab starting point.
TARGET_ROWS = 100_000
PAGE_SIZE = 25_000

OUTPUT_DIR = Path("nyc311_project_outputs")
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

SELECTED_COLUMNS = [
    "unique_key",
    "created_date",
    "closed_date",
    "agency",
    "complaint_type",
    "descriptor",
    "location_type",
    "incident_zip",
    "borough",
    "status",
    "due_date",
    "resolution_description",
]

TEXT_COLUMNS = [
    "agency",
    "complaint_type",
    "descriptor",
    "location_type",
    "borough",
    "status",
    "resolution_description",
]

DATE_COLUMNS = ["created_date", "closed_date", "due_date"]

CANONICAL_BOROUGHS = {
    "BRONX",
    "BROOKLYN",
    "MANHATTAN",
    "QUEENS",
    "STATEN ISLAND",
    "UNSPECIFIED",
}

CANONICAL_STATUSES = {
    "ASSIGNED",
    "CLOSED",
    "DRAFT",
    "IN PROGRESS",
    "OPEN",
    "PENDING",
    "STARTED",
    "UNSPECIFIED",
}


# ============================================================
# 2. DOWNLOAD DATA FROM NYC OPEN DATA
# ============================================================

def download_nyc311(
    target_rows: int = TARGET_ROWS,
    page_size: int = PAGE_SIZE,
    pause_seconds: float = 0.15,
) -> pd.DataFrame:
    """Download selected NYC 311 fields with Socrata pagination."""
    if target_rows <= 0:
        raise ValueError("target_rows must be greater than zero.")
    if page_size <= 0 or page_size > 50_000:
        raise ValueError("page_size must be between 1 and 50,000.")

    frames: list[pd.DataFrame] = []
    pages = math.ceil(target_rows / page_size)
    select_clause = ",".join(SELECTED_COLUMNS)

    print(f"Downloading up to {target_rows:,} records in {pages} page(s)...")

    for page in range(pages):
```

```
    for page in range(pages):
        offset = page * page_size
        remaining = target_rows - offset
        current_limit = min(page_size, remaining)

        params = {
            "$select": select_clause,
            "$limit": current_limit,
            "$offset": offset,
            # A deterministic order makes repeated runs more reproducible.
            "$order": "created_date DESC, unique_key DESC",
        }

        response = requests.get(API_URL, params=params, timeout=90)
        response.raise_for_status()
        records = response.json()

        if not records:
            print("No additional records were returned.")
            break

        frame = pd.DataFrame.from_records(records)
        frames.append(frame)
        print(f"  Page {page + 1}: {len(frame):,} records")

        if len(frame) < current_limit:
            break

        time.sleep(pause_seconds)

    if not frames:
        raise RuntimeError("The API returned no data.")

    raw = pd.concat(frames, ignore_index=True)

    # Ensure every expected field exists even if an API page omits it.
    for column in SELECTED_COLUMNS:
        if column not in raw.columns:
            raw[column] = pd.NA

    raw = raw[SELECTED_COLUMNS].head(target_rows)
    print(f"Downloaded shape: {raw.shape}")
    return raw


# ============================================================
# 3. INITIAL DATA PROFILE
# ============================================================

def profile_data(df: pd.DataFrame, name: str) -> pd.DataFrame:
    """Create a compact profiling table for presentation evidence."""
    profile = pd.DataFrame({
        "column": df.columns,
        "dtype": [str(df[col].dtype) for col in df.columns],
        "missing_count": [int(df[col].isna().sum()) for col in df.columns],
        "missing_percent": [
            round(float(df[col].isna().mean() * 100), 2) for col in df.columns
        ],
        "unique_values": [int(df[col].nunique(dropna=True)) for col in df.columns],
    })

    print(f"\n{name} DATASET")
    print("-" * 60)
    print(f"Rows: {len(df):,}")
    print(f"Columns: {df.shape[1]}")
    print(f"Exact duplicate rows: {df.duplicated().sum():,}")
    if "unique_key" in df.columns:
        print(f"Duplicate unique keys: {df['unique_key'].duplicated().sum():,}")

    return profile


# ============================================================
# 4. QUALITY SCORE FUNCTIONS
# ============================================================

def as_missing_aware(df: pd.DataFrame) -> pd.DataFrame:
```

```
    """Treat empty or whitespace-only strings as missing."""
    return df.replace(r"^\s*$", np.nan, regex=True)


def completeness_score(df: pd.DataFrame) -> float:
    """Average non-missing rate across fields essential to the AI use case."""
    required = [
        "unique_key",
        "created_date",
        "agency",
        "complaint_type",
        "borough",
        "status",
    ]
    work = as_missing_aware(df[required].copy())
    return round(float(work.notna().mean().mean() * 100), 2)


def uniqueness_score(df: pd.DataFrame) -> float:
    """Percent of rows represented by a unique service-request key."""
    if len(df) == 0:
        return 0.0
    keys = df["unique_key"].astype("string").str.strip()
    valid_keys = keys.notna() & keys.ne("")
    unique_valid = keys[valid_keys].nunique()
    return round(float(unique_valid / len(df) * 100), 2)


def validity_score(df: pd.DataFrame) -> float:
    """Average rate of records satisfying documented format/domain rules."""
    created = pd.to_datetime(df["created_date"], errors="coerce")
    closed = pd.to_datetime(df["closed_date"], errors="coerce")
    due = pd.to_datetime(df["due_date"], errors="coerce")

    borough = df["borough"].astype("string").str.strip().str.upper()
    zip_code = df["incident_zip"].astype("string").str.strip()
    key = df["unique_key"].astype("string").str.strip()

    rules = pd.DataFrame({
        "created_date_valid": created.notna(),
        "closed_date_valid": df["closed_date"].isna() | closed.notna(),
        "due_date_valid": df["due_date"].isna() | due.notna(),
        "date_order_valid": closed.isna() | created.isna() | (closed >= created),
        "borough_valid": borough.isin(CANONICAL_BOROUGHS),
        "zip_valid": zip_code.isna() | zip_code.str.match(r"^\d{5}(?:-\d{4})?$", na=False),
        "key_valid": key.str.match(r"^\d+$", na=False),
    })
    return round(float(rules.mean().mean() * 100), 2)


def consistency_score(df: pd.DataFrame) -> float:
    """Average rate of canonical formatting across key fields."""
    complaint = df["complaint_type"].astype("string")
    agency = df["agency"].astype("string")
    borough = df["borough"].astype("string")
    status = df["status"].astype("string")

    rules = pd.DataFrame({
        "complaint_trimmed": complaint.isna() | complaint.eq(complaint.str.strip()),
        "agency_trimmed": agency.isna() | agency.eq(agency.str.strip()),
        "borough_canonical": borough.isna() | borough.str.strip().str.upper().isin(CANONICAL_BOROUGHS),
        "status_canonical": status.isna() | status.str.strip().str.upper().isin(CANONICAL_STATUSES),
        "no_double_spaces": complaint.isna() | ~complaint.str.contains(r"\s{2,}", regex=True, na=False),
    })
    return round(float(rules.mean().mean() * 100), 2)


def accuracy_proxy_score(df: pd.DataFrame) -> float:
    """
    Internal logical accuracy proxy.

    True accuracy requires external ground truth. This proxy checks whether:
    - closed dates do not precede creation dates,
    - records marked Closed have a closed date,
    - records marked Closed usually have resolution text.
    """
    created = pd.to_datetime(df["created_date"], errors="coerce")
```

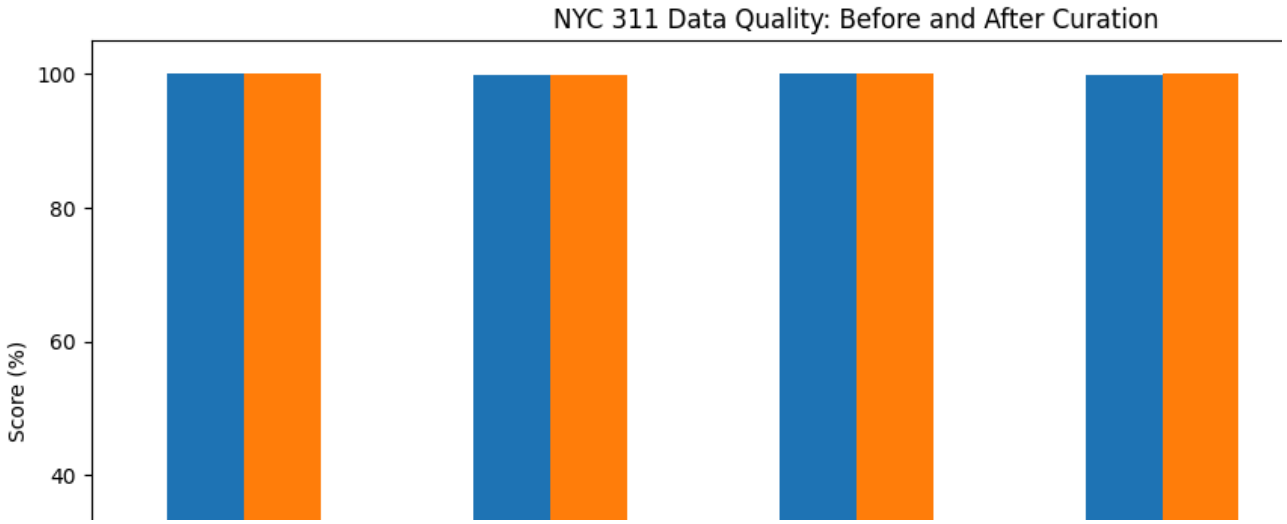
```
    created = pd.to_datetime(df["created_date"], errors="coerce")
    closed = pd.to_datetime(df["closed_date"], errors="coerce")
    status = df["status"].astype("string").str.strip().str.upper()
    resolution = df["resolution_description"].astype("string").str.strip()

    closed_status = status.eq("CLOSED")
    rules = pd.DataFrame({
        "chronology_logical": closed.isna() | created.isna() | (closed >= created),
        "closed_status_has_date": ~closed_status | closed.notna(),
        "closed_status_has_resolution": ~closed_status | resolution.notna() & resolution.ne(""),
    })
    return round(float(rules.mean().mean() * 100), 2)


def calculate_quality_scores(df: pd.DataFrame) -> dict[str, float]:
    return {
        "Completeness": completeness_score(df),
        "Accuracy (proxy)": accuracy_proxy_score(df),
        "Consistency": consistency_score(df),
        "Validity": validity_score(df),
        "Uniqueness": uniqueness_score(df),
    }


# ============================================================
# 5. CURATION PIPELINE
# ============================================================

def normalize_spaces(series: pd.Series) -> pd.Series:
    return (
        series.astype("string")
        .str.strip()
        .str.replace(r"\s+", " ", regex=True)
    )


def curate_data(raw: pd.DataFrame) -> pd.DataFrame:
    """Apply documented cleaning, standardization, and privacy decisions."""
    df = raw.copy()

    # Standardize missing representations.
    df = df.replace(
        ["", " ", "N/A", "NA", "NULL", "null", "None", "UNKNOWN"],
        pd.NA,
    )

    # Trim and collapse repeated whitespace.
    for column in TEXT_COLUMNS:
        df[column] = normalize_spaces(df[column])

    # Canonicalize fields with known categories.
    df["agency"] = df["agency"].str.upper()
    df["borough"] = df["borough"].str.upper().fillna("UNSPECIFIED")
    df["status"] = df["status"].str.upper().fillna("UNSPECIFIED")

    # Keep descriptive fields usable without pretending missing text is factual.
    df["descriptor"] = df["descriptor"].fillna("Not Provided")
    df["location_type"] = df["location_type"].fillna("Not Specified")
    df["resolution_description"] = df["resolution_description"].fillna(
        "Not Available"
    )

    # Parse dates.
    for column in DATE_COLUMNS:
        df[column] = pd.to_datetime(df[column], errors="coerce")

    # Convert keys to a clean string and remove invalid/duplicate service requests.
    df["unique_key"] = df["unique_key"].astype("string").str.strip()
    df = df[df["unique_key"].str.match(r"^\d+$", na=False)].copy()
    df = df.sort_values(["created_date", "unique_key"], ascending=[False, False])
    df = df.drop_duplicates(subset=["unique_key"], keep="first")

    # Remove impossible date relationships by setting the questionable value to missing.
    invalid_closed = (
        df["closed_date"].notna()
        & df["created_date"].notna()
        & (df["closed_date"] < df["created_date"])
```

```
    )
    df.loc[invalid_closed, "closed_date"] = pd.NaT

    # Privacy-aware location transformation:
    # Retain borough for geographic analysis, generalize ZIP to three digits,
    # then remove the full ZIP code from the curated output.
    zip_clean = (
        df["incident_zip"]
        .astype("string")
        .str.extract(r"(\d{5})", expand=False)
    )
    df["zip3_generalized"] = zip_clean.str[:3].fillna("Unknown")
    df = df.drop(columns=["incident_zip"])

    # Derived field useful for analysis.
    df["resolution_time_hours"] = (
        (df["closed_date"] - df["created_date"]).dt.total_seconds() / 3600
    )
    df.loc[df["resolution_time_hours"] < 0, "resolution_time_hours"] = np.nan

    return df.reset_index(drop=True)


# ============================================================
# 6. PRELIMINARY URGENCY ANNOTATION
# ============================================================

HIGH_PATTERNS = [
    r"\bgas\b",
    r"\bsmoke\b",
    r"\bsewer backup\b",
    r"\bstreet flood",
    r"\bwater main\b",
    r"\bsinkhole\b",
    r"\bcollaps",
    r"\bexposed wire\b",
    r"\btraffic signal\b",
    r"\bno heat\b",
    r"\bdangerous building\b",
]

MEDIUM_PATTERNS = [
    r"\brodent\b",
    r"\brat\b",
    r"\bpothole\b",
    r"\bblocked driveway\b",
    r"\bmissed collection\b",
    r"\bstreetlight\b",
    r"\bnoise\b",
    r"\billegal parking\b",
    r"\bunsanitary\b",
]

LOW_PATTERNS = [
    r"\bgraffiti\b",
    r"\blitter\b",
    r"\bpainting\b",
    r"\bpruning\b",
    r"\binformation\b",
    r"\bsign\b",
    r"\brecycling\b",
]


def contains_any(text: str, patterns: Iterable[str]) -> str | None:
    for pattern in patterns:
        if re.search(pattern, text, flags=re.IGNORECASE):
            return pattern
    return None


def suggest_urgency(row: pd.Series) -> tuple[str, str]:
    """
    Create a preliminary rule-based label.

    These rules are for annotation support only and require human review.
    NYC 311 is a non-emergency service; emergencies should be directed to 911
```

```
    """
    combined = " | ".join(
        str(row.get(col, "") or "")
        for col in ["complaint_type", "descriptor", "resolution_description"]
    ).lower()

    match = contains_any(combined, HIGH_PATTERNS)
    if match:
        return "High", f"Matched high-priority rule: {match}"

    match = contains_any(combined, MEDIUM_PATTERNS)
    if match:
        return "Medium", f"Matched medium-priority rule: {match}"

    match = contains_any(combined, LOW_PATTERNS)
    if match:
        return "Low", f"Matched low-priority rule: {match}"

    return "Needs Review", "No rule matched; human review required"


def add_preliminary_labels(df: pd.DataFrame) -> pd.DataFrame:
    labeled = df.copy()
    suggestions = labeled.apply(suggest_urgency, axis=1, result_type="expand")
    suggestions.columns = ["service_urgency_label", "reason_for_label"]
    return pd.concat([labeled, suggestions], axis=1)


def create_annotation_sample(
    df: pd.DataFrame,
    sample_size: int = 50,
    random_state: int = 42,
) -> pd.DataFrame:
    """Create a reproducible 50-record review sheet."""
    n = min(sample_size, len(df))
    columns = [
        "unique_key",
        "created_date",
        "agency",
        "complaint_type",
        "descriptor",
        "borough",
        "status",
        "service_urgency_label",
        "reason_for_label",
    ]
    sample = df.sample(n=n, random_state=random_state)[columns].copy()
    sample["human_review_label"] = ""
    sample["reviewer_notes"] = ""
    return sample


# ============================================================
# 7. VISUALIZATION AND EXPORT
# ============================================================

def make_quality_chart(score_table: pd.DataFrame, output_path: Path) -> None:
    ax = score_table.plot(
        x="Quality Dimension",
        y=["Before", "After"],
        kind="bar",
        figsize=(11, 6),
    )
    ax.set_title("NYC 311 Data Quality: Before and After Curation")
    ax.set_xlabel("Quality Dimension")
    ax.set_ylabel("Score (%)")
    ax.set_ylim(0, 105)
    ax.tick_params(axis="x", rotation=20)
    ax.legend(title="")
    plt.tight_layout()
    plt.savefig(output_path, dpi=200, bbox_inches="tight")
    plt.show()


def main() -> None:
    raw = download_nyc311()
```

```
    raw.to_csv(OUTPUT_DIR / "nyc311_raw_subset.csv", index=False)
    raw_profile = profile_data(raw, "RAW")
    raw_profile.to_csv(OUTPUT_DIR / "raw_profile.csv", index=False)

    before_scores = calculate_quality_scores(raw)
    print("\nBefore curation scores:")
    print(before_scores)

    curated = curate_data(raw)
    curated = add_preliminary_labels(curated)

    curated_profile = profile_data(curated, "CURATED")
    curated_profile.to_csv(OUTPUT_DIR / "curated_profile.csv", index=False)

    # Score only the original/shared fields required by the score functions.
    # Recreate incident_zip as missing because the curated file intentionally
    # removes the full ZIP for privacy. Validity then evaluates the remaining
    # rules and treats ZIP as unavailable.
    score_ready = curated.copy()
    score_ready["incident_zip"] = pd.NA

    after_scores = calculate_quality_scores(score_ready)
    print("\nAfter curation scores:")
    print(after_scores)

    score_table = pd.DataFrame({
        "Quality Dimension": list(before_scores.keys()),
        "Before": list(before_scores.values()),
        "After": [after_scores[key] for key in before_scores],
    })
    score_table["Change"] = (
        score_table["After"] - score_table["Before"]
    ).round(2)

    print("\nQUALITY SCORE TABLE")
    print(score_table.to_string(index=False))

    score_table.to_csv(
        OUTPUT_DIR / "quality_scores_before_after.csv",
        index=False,
    )

    make_quality_chart(
        score_table,
        OUTPUT_DIR / "quality_scores_before_after.png",
    )

    curated.to_csv(
        OUTPUT_DIR / "nyc311_curated_with_preliminary_labels.csv",
        index=False,
    )

    annotation_sample = create_annotation_sample(curated)
    annotation_sample.to_csv(
        OUTPUT_DIR / "annotation_review_sample_50.csv",
        index=False,
    )

    print("\nUrgency label distribution:")
    print(
        curated["service_urgency_label"]
        .value_counts(dropna=False)
        .to_string()
    )

    print("\nFiles saved to:", OUTPUT_DIR.resolve())
    for file in sorted(OUTPUT_DIR.iterdir()):
        print(" -", file.name)


if __name__ == "__main__":
    main()

```

```
Downloading up to 100,000 records in 4 page(s)...
  Page 1: 25,000 records
  Page 2: 25,000 records
  Page 3: 25,000 records
  Page 4: 25,000 records
Downloaded shape: (100000, 12)

RAW DATASET
------------------------------------------------------------
Rows: 100,000
Columns: 12
Exact duplicate rows: 0
Duplicate unique keys: 0

Before curation scores:
{'Completeness': 100.0, 'Accuracy (proxy)': 99.96, 'Consistency': 100.0, 'Validity': 99.99, 'Uniqueness': 100.0
------------------------------------
Exact duplicate rows: 0
After curation scores:
oxy)': 99.98, 'Consistency': 100.0, 'Validity': 100.0, 'Uniqueness': 100.0

QUALITY SCORE TABLE
      Quality Dimension  Before  After  Change
           Completeness  100.00 100.00    0.00
       Accuracy (proxy)   99.96  99.98    0.02
            Consistency  100.00 100.00    0.00
               Validity   99.99 100.00    0.01
             Uniqueness  100.00 100.00    0.00
```



Files produced

After the final cell runs, open the `nyc311_project_outputs` folder. It will contain:

- `nyc311_raw_subset.csv`
- `raw_profile.csv`
- `curated_profile.csv`
- `quality_scores_before_after.csv`
- `quality_scores_before_after.png`
- `nyc311_curated_with_preliminary_labels.csv`
- `annotation_review_sample_50.csv`

Use the score table and PNG chart on the **Quality Results** slide.

Use the annotation sample as evidence for the **Pipeline** and **Reflection** sections.